

# ResQ

## AI-Powered Emergency Response System for Indian Roads

National Road Safety Hackathon 2026 | Track: RoadSoS

### Team ResQ

Amit • Hataf Didi • Chandrashekhar • Kamal • Mohak

May 2026



## Table of Contents

---

1. Project Overview 3
2. Tech Stack (Software Packages) 4
3. Hardware Components 5
4. Assumptions 6
5. Code Structure 7
6. Complete Code 8
7. Setup Instructions 12
8. Future Scope 13



## 1. Project Overview

---

ResQ is an AI-powered emergency response system designed for Indian roads. It combines:

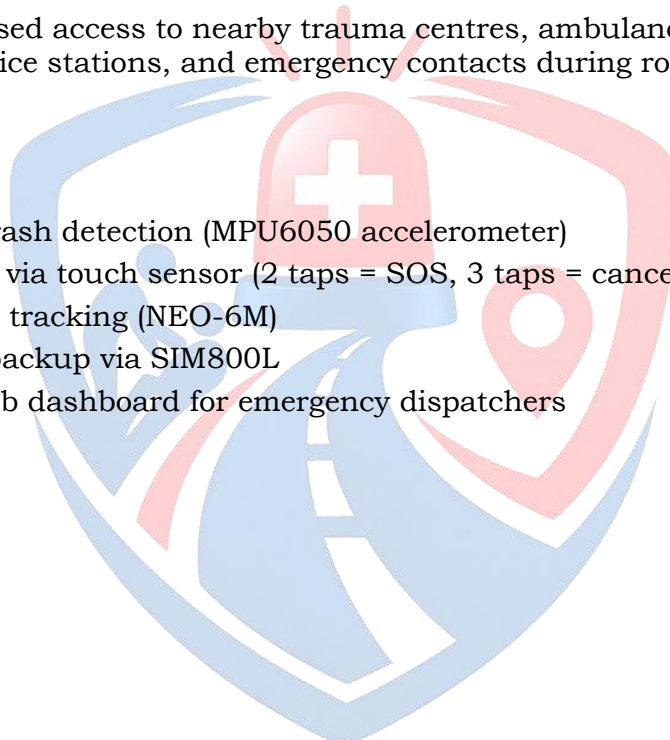
- Mobile App – One-tap SOS + automatic crash detection
- Web Dashboard – Real-time command center for emergency services
- ESP32 Hardware Module – In-vehicle crash detection without phone dependency

### Problem Statement (RoadSoS)

Provide location-based access to nearby trauma centres, ambulance services, vehicle rescue services, police stations, and emergency contacts during road accidents.

### ResQ Solution

- Automatic crash detection (MPU6050 accelerometer)
- Manual SOS via touch sensor (2 taps = SOS, 3 taps = cancel)
- GPS location tracking (NEO-6M)
- SMS + Call backup via SIM800L
- Real-time web dashboard for emergency dispatchers



## 2. Tech Stack (Software Packages)

### 2.1 ESP32 (Hardware Code)

| Package                 | Version  | Purpose              |
|-------------------------|----------|----------------------|
| Wire                    | Built-in | I2C communication    |
| Adafruit GFX            | 1.11.5   | OLED graphics        |
| Adafruit SSD1306        | 2.5.7    | OLED display driver  |
| Adafruit MPU6050        | 2.2.0    | Accelerometer sensor |
| Adafruit Unified Sensor | 1.1.6    | Sensor library       |
| TinyGPSPlus             | 1.0.3    | GPS data parsing     |
| SD                      | Built-in | SD card logging      |
| SPI                     | Built-in | SPI communication    |

### 2.2 Web Dashboard (Frontend)

| Package          | Version | Purpose           |
|------------------|---------|-------------------|
| React            | 18.2.0  | UI framework      |
| TypeScript       | 5.2.0   | Type safety       |
| Vite             | 5.0.0   | Build tool        |
| Tailwind CSS     | 3.4.0   | Styling           |
| shadcn/ui        | Latest  | UI components     |
| Socket.io-client | 4.5.4   | Real-time updates |
| Leaflet          | 1.9.0   | Maps              |
| React Leaflet    | 4.2.0   | React map wrapper |
| Lucide React     | 0.263.0 | Icons             |

### 2.3 Backend

| Package     | Version | Purpose      |
|-------------|---------|--------------|
| Node.js     | 18.x    | Runtime      |
| Express     | 4.18.2  | API server   |
| Socket.io   | 4.5.4   | WebSocket    |
| Drizzle ORM | 0.29.0  | Database ORM |

| Package    | Version | Purpose     |
|------------|---------|-------------|
| PostgreSQL | 14.x    | Database    |
| TypeScript | 5.2.0   | Type safety |
| Zod        | 3.22.0  | Validation  |



### 3. Hardware Components

| Component                 | Quantity | Purpose                                 |
|---------------------------|----------|-----------------------------------------|
| ESP32 (38-pin)            | 1        | Main microcontroller                    |
| MPU6050                   | 1        | Crash detection (accelerometer)         |
| SIM800L                   | 1        | SMS + Voice call backup                 |
| NEO-6M GPS                | 1        | Location tracking                       |
| Touch Sensor              | 1        | Manual SOS (2 tap = ON, 3 tap = CANCEL) |
| OLED Display (128x64 I2C) | 1        | Status display                          |
| Micro SD Card Module      | 1        | Data logging                            |
| Battery Pack (3.7V–5V)    | 1        | Power backup                            |

#### Pin Connection Table

| Component        | ESP32 Pin |
|------------------|-----------|
| MPU6050 SDA      | GPIO 21   |
| MPU6050 SCL      | GPIO 22   |
| OLED SDA         | GPIO 21   |
| OLED SCL         | GPIO 22   |
| SIM800L TX       | GPIO 16   |
| SIM800L RX       | GPIO 17   |
| GPS TX           | GPIO 18   |
| GPS RX           | GPIO 19   |
| Touch Sensor SIG | GPIO 4    |
| SD Card CS       | GPIO 5    |
| SD Card MOSI     | GPIO 23   |
| SD Card MISO     | GPIO 19   |
| SD Card SCK      | GPIO 18   |

## 4. Assumptions

### 4.1 Technical Assumptions

| # | Assumption                                            | Justification                     |
|---|-------------------------------------------------------|-----------------------------------|
| 1 | ESP32 has stable power supply                         | Battery backup ensures operation  |
| 2 | SIM800L has active SIM with SMS/call                  | Prepaid/postpaid SIM with balance |
| 3 | GPS works outdoors                                    | Standard limitation               |
| 4 | Touch sensor is accessible to driver/bystander        | Mounted on dashboard              |
| 5 | Internet available for dashboard (optional for ESP32) | Dashboard needs internet          |

### 4.2 Functional Assumptions

| # | Assumption                                  | Justification                    |
|---|---------------------------------------------|----------------------------------|
| 1 | 2 taps within 1 second = manual SOS         | User intention confirmed         |
| 2 | 3 taps within cancellation window = cancel  | Prevents false alarms            |
| 3 | MPU6050 threshold 2.5G = crash              | Calibrated for typical accidents |
| 4 | 15-second cancellation window is sufficient | User has time to cancel          |
| 5 | Emergency number is pre-configured          | User sets before use             |

### 4.3 Deployment Assumptions

| # | Assumption                                 | Justification               |
|---|--------------------------------------------|-----------------------------|
| 1 | Backend deployed on cloud (Render/Railway) | Public accessible           |
| 2 | Database is PostgreSQL                     | Persistent storage          |
| 3 | Dashboard accessible via browser           | No app installation needed  |
| 4 | ESP32 has WiFi credentials stored          | For dashboard communication |

## 5. Code Structure

---

```
resq/
├── mobile-app/
│   ├── lib/           # Flutter source code
│   ├── android/       # Android-specific config
│   ├── ios/           # iOS-specific config
│   └── pubspec.yaml
├── dashboard/
│   ├── public/        # Static files
│   ├── src/
│   │   ├── components/ # UI components
│   │   ├── pages/      # Dashboard pages
│   │   ├── services/   # API calls
│   │   └── styles/     # CSS modules
│   ├── server/
│   │   ├── models/    # MongoDB models
│   │   ├── routes/    # API routes
│   │   └── controllers/ # Business logic
│   └── package.json
├── hardware/
│   └── esp32/
│       ├── resq_edge.ino # Main ESP32 firmware
│       └── libraries/    # Required libraries
├── docs/
│   ├── api/            # API documentation
│   └── architecture/   # System design docs
├── .env.example
├── .gitignore
└── README.md
```



## 6. Complete Code

---

### 6.1 ESP32 Code — `resq_edge.ino`

Track: RoadSoS | Hackathon 2026

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <TinyGPSPlus.h>
#include <SD.h>
#include <SPI.h>

// OLED Display
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// MPU6050
Adafruit_MPU6050 mpu;

// GPS on Serial2
TinyGPSPlus gps;
#define GPS_RX 18
#define GPS_TX 19

// SIM800L on Serial1
#define SIM800L_RX 16
#define SIM800L_TX 17

// Touch Sensor
#define TOUCH_SENSOR_PIN 4

// SD Card
#define SD_CS 5

// Configuration
#define ACCEL_THRESHOLD 2.5
#define SOS_CANCEL_WINDOW 15000
#define EMERGENCY_NUMBER "+91XXXXXXXXXX"
```

```
// Global Variables
unsigned long crashDetectedTime = 0;
bool sosTriggered = false;
bool sosCancelled = false;
int tapCount = 0;
unsigned long lastTapTime = 0;
bool cancellationWindowActive = false;
String lastValidLocation = "0.000000,0.000000";

void setup() {
  Serial.begin(115200);
  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED init failed");
  }
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);
  display.println("ResQ Edge v2.0");
  display.display();
  delay(2000);
  if (!mpu.begin()) {
    Serial.println("MPU6050 not found");
  } else {
    mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  }
  if (!SD.begin(SD_CS)) {
    Serial.println("SD Card not found");
  } else {
    logToSD("ResQ Edge Started");
  }
  Serial2.begin(9600, SERIAL_8N1, GPS_RX, GPS_TX);
  Serial1.begin(9600, SERIAL_8N1, SIM800L_RX, SIM800L_TX);
  delay(1000);
  Serial1.println("AT");
  delay(500);
  pinMode(TOUCH_SENSOR_PIN, INPUT_PULLUP);
  display.clearDisplay();
  display.setCursor(0, 0);
  display.println("ResQ Ready");
  display.println("2 tap = SOS");
  display.println("3 tap = Cancel");
  display.display();
}
```

```
}

void loop() {
  checkTouchSensor();
  checkMPU6050();
  readGPS();
  handleCancellationWindow();
  updateDisplay();
  delay(10);
}

void checkTouchSensor() {
  int touchState = digitalRead(TOUCH_SENSOR_PIN);
  if (touchState == LOW) {
    unsigned long now = millis();
    if (now - lastTapTime > 300) {
      tapCount++;
      lastTapTime = now;
      if (tapCount == 2 && !sosTriggered && !cancellationWindowActive) {
        crashDetectedTime = millis();
        cancellationWindowActive = true;
        sosCancelled = false;
        display.clearDisplay();
        display.setCursor(0, 0);
        display.println("SOS in 15s");
        display.println("Tap 3 to cancel");
        display.display();
      }
      if (tapCount == 3 && cancellationWindowActive) {
        sosCancelled = true;
        cancellationWindowActive = false;
        tapCount = 0;
        display.clearDisplay();
        display.setCursor(0, 0);
        display.println("SOS Cancelled");
        display.display();
        delay(1000);
      }
    }
    if (millis() - lastTapTime > 1000) { tapCount = 0; }
  }
}

void checkMPU6050() {
```

```
sensors_event_t a, g, temp;
mpu.getEvent(&a, &g, &temp);
float accelMagnitude = sqrt(a.acceleration.x * a.acceleration.x +
                             a.acceleration.y * a.acceleration.y +
                             a.acceleration.z * a.acceleration.z);
float gForce = accelMagnitude / 9.8;
if (gForce > ACCEL_THRESHOLD && !sosTriggered && !cancellationWindowActive) {
    crashDetectedTime = millis();
    cancellationWindowActive = true;
    sosCancelled = false;
    logToSD("Crash detected: " + String(gForce) + "G");
    display.clearDisplay();
    display.setCursor(0, 0);
    display.println("CRASH DETECTED");
    display.display();
}
}

void readGPS() {
    while (Serial2.available() > 0) {
        char c = Serial2.read();
        if (gps.encode(c)) {
            if (gps.location.isValid()) {
                lastValidLocation = String(gps.location.lat(), 6) + "," + String(gps.location.lng(), 6);
            }
        }
    }
}

void handleCancellationWindow() {
    if (cancellationWindowActive && !sosTriggered && !sosCancelled) {
        if (millis() - crashDetectedTime > SOS_CANCEL_WINDOW) {
            triggerSOS();
        }
    }
}

void triggerSOS() {
    sosTriggered = true;
    cancellationWindowActive = false;
    logToSD("SOS TRIGGERED at: " + lastValidLocation);
    display.clearDisplay();
    display.setCursor(0, 0);
    display.println("SOS TRIGGERED!");
}
```

```
display.println("Help coming");
display.display();
sendSMS();
sendToDashboard();
}

void sendSMS() {
  Serial.println("AT+CMGF=1");
  delay(500);
  Serial.print("AT+CMGS=\"");
  Serial.print(EMERGENCY_NUMBER);
  Serial.println("\"");
  delay(500);
  Serial.print("RESQ SOS! Location: ");
  Serial.print(lastValidLocation);
  Serial.write(26);
  delay(500);
  logToSD("SMS sent");
}

void sendToDashboard() {
  logToSD("Data sent to dashboard: " + lastValidLocation);
}

void logToSD(String message) {
  File logFile = SD.open("/log.txt", FILE_APPEND);
  if (logFile) {
    logFile.print(millis());
    logFile.print(": ");
    logFile.println(message);
    logFile.close();
  }
}

void updateDisplay() {
  static unsigned long lastUpdate = 0;
  if (millis() - lastUpdate > 1000) {
    display.clearDisplay();
    display.setCursor(0, 0);
    if (sosTriggered) {
      display.println("SOS ACTIVE");
      display.println("Help on way");
    } else if (cancellationWindowActive) {
```

```
int remaining = (SOS_CANCEL_WINDOW - (millis() - crashDetectedTime)) / 1000;
display.print("Cancel: ");
display.print(remaining);
display.println("s");
} else {
display.println("ResQ Ready");
display.println("2 tap = SOS");
display.print("GPS: ");
display.println(gps.location.isValid() ? "OK" : "Wait");
}
display.display();
lastUpdate = millis();
}
}
```

## 6.2 Dashboard Code

- dashboard.html — Provided separately
- dashboard.css — Provided separately
- dashboard.js — Provided separately

## 6.3 Backend Code

- server.js — Provided separately
- package.json — Provided separately

## 7. Setup Instructions

---

### 7.1 ESP32 Setup

- Install Arduino IDE
- Install required libraries (see Section 2.1)
- Select board: ESP32 Dev Module
- Connect ESP32 via USB
- Upload resq\_edge.ino

### 7.2 Backend Setup

```
cd backend
npm install
npm start
```

### 7.3 Dashboard Setup

```
cd dashboard
# Open dashboard.html in browser
# Or serve via live server
```

### 7.4 Environment Variables

Create a .env file with the following variables:

```
DATABASE_URL=postgresql://localhost:5432/resq
PORT=3000
EMERGENCY_NUMBER=+91XXXXXXXXXX
```

## 8. Future Scope

| Feature                     | Description                              |
|-----------------------------|------------------------------------------|
| Mobile App Integration      | Flutter/React Native app for iOS/Android |
| Live Map Integration        | Google Maps API for real-time tracking   |
| AI Prediction               | ML model for accident prediction         |
| Multiple Emergency Contacts | Group SMS/WhatsApp alerts                |
| Voice Assistant Integration | Alexa/Google Assistant support           |





## Declaration

---

We hereby declare that this submission is our original work and has been developed for the National Road Safety Hackathon 2026, IIT Madras.

| Name           | Role                             |
|----------------|----------------------------------|
| Amit           | Project Director & Hardware Lead |
| Hataf Didi     | App Development Lead             |
| Chandrashekhar | Dashboard & Backend Lead         |
| Kamal          | Pitch & Presentation Lead        |
| Mohak          | Mechanical Research Lead         |

**Date: May 2026**

**Track: RoadSoS**

**Github link :**

[https://github.com/AmitKumarPrasad1846/ResQ\\_RoadSoS\\_IITM.git](https://github.com/AmitKumarPrasad1846/ResQ_RoadSoS_IITM.git)

**PPT link :**

<https://canva.link/gjhgkm648a54bro>

**Complete File link :**

<https://drive.google.com/drive/folders/1K-GvJt410Go-Tt5r01g2fRjFEG6wTsdX?usp=sharing>